Московский авиационный институт (национальный исследовательский университет)

Факультет информационных технологий и прикладной математики

Кафедра вычислительной математики и программирования

Лабораторная работа №1 по курсу «Информационный поиск»

Студент: П. А. Земсков
Преподаватель: А. А. Кухтичев
Группа: М8О-401Б-23

Дата: 05.10.2026
Оценка:
Подпись:

Москва, 2026

Лабораторная работа №1 «Добыча корпуса документов»

Необходимо подготовить корпус документов, который будет использован при выполнении остальных лабораторных работ:

- Скачать его к себе на компьютер. В отчёте нужно указать источник данных.
- Ознакомиться с ним, изучить его характеристики. Из чего состоит текст? Есть ли дополнительная мета-информация? Если разметка текста, какая она?
- Разбить на документы.
- Выделить текст.
- Найти существующие поисковики, которые уже можно использовать для поиска по выбранному набору документов (встроенный поиск Википедии, поиск Google с использованием ограничений на URL или на сайт). Если такого поиска найти невозможно, то использовать корпус для выполнения лабораторных работ нельзя!
- Привести несколько примеров запросов к существующим поисковикам, указать недостатки в полученной поисковой выдаче.

В результатах работы должна быть указаны статистическая информация о корпусе:

- Размер «сырых» данных.
- Количество документов.
- Размер текста, выделенного из «сырых» данных.
- Средний размер документа, средний объём текста в документе.

1 Описание

1 Источник данных

Корпус построен из статей русской Википедии (`ru.wikipedia.org`). Статьи получаются через открытый MediaWiki Action API: метод `list=random` (с фильтрами: основное пространство имён, без перенаправлений) возвращает случайные статьи, метод `action=parse` — HTML-код статьи. Ключ API и авторизация не нужны; запросы отправляются с идентифицирующим `User-Agent`, как требуют правила Викимедиа.

2 Характеристики корпуса

Документ — «сырая» HTML-страница статьи (вёрстка MediaWiki: заголовки `h2`/`h3`, абзацы `p`, списки, таблицы, инфобоксы, навигационные блоки, ссылки на источники) плюс JSON-файл с метаданными: url, источник, идентификатор и заголовок статьи, очищенный текст, число символов и слов, время загрузки (Unix time stamp). Язык — русский (доля кириллицы среди букв ≈94,2%, остальное — латиница в названиях и терминах). Тематика разнообразная: населённые пункты, персоналии, биология, техника, спорт, история и т.д.

3 Разбиение на документы

Принцип «одна статья — один документ». Файлы хранятся как `corpus/wikipedia/<pageid>.html` и `<pageid>.json`, где `pageid` — идентификатор статьи в Википедии. Повторный запуск программы пропускает уже скачанные статьи. Статьи короче 1000 символов (заготовки) в корпус не включаются.

4 Выделение текста

Из HTML текст выделяется библиотекой BeautifulSoup. Сначала удаляются служебные элементы: `style`, таблицы, сноски (`sup.reference`), списки источников, навигационные блоки (`.navbox`), пометки `.hatnote`, миниатюры изображений и ссылки «править». Затем собирается текст абзацев, заголовков и элементов списков.

5 Существующие поисковики

Для поиска по выбранным документам уже можно использовать:

- встроенный поиск Википедии (Служебная:Поиск и API `list=search`);
- Google и Яндекс с ограничением по сайту: `site:ru.wikipedia.org`.

6 Примеры запросов и недостатки выдачи

Запросы выполнены через встроенный поиск Википедии (API `list=search`).

1. `компьютер мышь` — найдено 1864 статьи; первой идёт «Компьютерная мышь», но следом — страница значений «Мышь (значения)» и общие статьи («Периферийное устройство», «Персональный компьютер»), которые лишь упоминают оба слова.

2. `«информационный поиск»` (точная фраза) — найдено 65 статей; первой идёт «Информационный поиск», дальше — статьи по смежным темам (кластеризация документов, метапоиск). Ранжирование по релевантности не объясняется, пользователь не видит, почему статья оказалась выше.

3. `яблоко` — найдено 8857 статей; в начале выдачи плоды, политическая партия «Яблоко», сорт «Золотое яблоко» и анатомический термин «Адамово яблоко»: многозначность слова не разрешается, нужно уточнять запрос вручную.

Общие недостатки: нет учёта морфологии и синонимов в явном виде, не разрешается многозначность, нет возможности настраивать ранжирование, а внешние поисковики (Google, Яндекс) возвращают страницы вне нашего корпуса.

7 Статистика по корпусу

Статистика получена командой `python3 main.py --stats` по полностью скачанному корпусу.

Показатель	Значение
Размер «сырых» данных	2074 МБ (≈2,0 ГБ)
Количество документов	30027
Размер выделенного текста	231,5 МБ
Средний размер документа	69,1 КБ
Средний объём текста в документе	7,7 КБ
Среднее число слов в документе	639

2 Исходный код

Программа `main.py` написана на Python 3 и использует библиотеки `requests` и `BeautifulSoup`. Основные функции: `api` (запрос к MediaWiki API с повторами), `random_pages` (список случайных статей), `extract_text` (очистка HTML), `download` (загрузка и сохранение одной статьи) и `crawl` (основной цикл).

Этапы работы:

1. получение пачки из 500 случайных статей (без перенаправлений);
2. отбрасывание уже скачанных идентификаторов;
3. параллельная (в нескольких потоках) загрузка HTML каждой статьи;
4. выделение текста, пропуск коротких статей;
5. сохранение HTML-документа и JSON-метаданных.

Учтены особенности реального API: временные сбои (повторные попытки), ограничение частоты запросов (код 429 и заголовок `Retry-After`), пропуск удалённых и недоступных статей, возможность остановить и продолжить загрузку.

Запрос к API с повторами и обработкой ограничения частоты:

```
def api(**params):
    for attempt in range(5):
        try:
            r = session.get(API, params={"format": "json",
                            "formatversion": 2, **params}, timeout=30)
            if r.status_code == 200:
                return r.json()
            if r.status_code == 429: # rate limited
                time.sleep(int(r.headers.get("Retry-After", 10)))
                continue
        except (requests.RequestException, ValueError):
            pass
        time.sleep(5 * (attempt + 1))
    raise RuntimeError(f"API failed: {params}")
```

Выделение текста из HTML статьи:

```
def extract_text(html):
    soup = BeautifulSoup(html, "html.parser")
    for tag in soup.select(JUNK):
        tag.decompose()
    return "\n".join(t.get_text(" ", strip=True)
                     for t in soup.select("p, h2, h3, li")
                     if t.get_text(strip=True))
```

Основной цикл (пачки случайных статей, параллельная загрузка):

```
with ThreadPoolExecutor(workers) as pool:
    while count < limit:
        batch = [p for p in random_pages() if p[0] not in seen]
        seen.update(p[0] for p in batch)
        count += sum(pool.map(lambda p: download(p, folder), batch))
```

3 Выводы

Выполнив первую лабораторную работу по курсу «Информационный поиск», я научился собирать корпус документов из открытого API. Основное, что я узнал нового:

- работа с MediaWiki API: выбор случайных статей, получение HTML, разбор ответов в формате JSON;
- выделение текста из HTML по CSS-селекторам и удаление служебных элементов (таблиц, сносок, навигации);
- обработка ограничений при загрузке большого объёма данных: повторные попытки, учёт кода 429 и `Retry-After`, возможность продолжить загрузку. На практике при слишком большом числе потоков (8) API начал отказывать, а при попытке обкачать новостной сайт соединение было заблокировано целиком, поэтому источник был заменён на Википедию;
- подсчёт статистики корпуса: «сырой» HTML в 9 раз больше полезного текста (69,1 КБ против 7,7 КБ на документ).

Корпус из 30027 статей будет использован в следующих лабораторных работах для построения поискового робота, индекса и поиска. Встроенный поиск Википедии показал недостатки — неразрешённую многозначность, неочевидное ранжирование и выдачу общих статей вместо точных; их предстоит учесть при построении собственного поисковика.

Список литературы

[1] Маннинг, Рагхаван, Шютце *Введение в информационный поиск* — Издательский дом «Вильямс», 2011. Перевод с английского: доктор физ.-мат. наук Д. А. Клюшина — 528 с. (ISBN 978-5-8459-1623-4 (рус.))

[2] Список использованных источников оформлять нужно по ГОСТ Р 7.05-2008

[3] Русская Википедия // https://ru.wikipedia.org

[4] MediaWiki Action API // https://www.mediawiki.org/wiki/API:Main_page

[5] Requests: HTTP for Humans // https://docs.python-requests.org

[6] Beautiful Soup Documentation // https://www.crummy.com/software/BeautifulSoup/bs4/doc/